

A Fun SQL Detective Story (with Real Query Outputs)

Imagine a museum where paintings are made not of paint, but of glowing pixels on massive screens. Welcome to the **Pixel Art Museum** — home of the legendary **Pixel Mona Lisa**.

One Friday night at 10:15 PM, disaster struck:



FILE NOT FOUND

The museum's most valuable digital artwork vanished without a trace. No broken glass. No alarms. Only data. As the lead **Database Detective**, our mission was clear: solve the mystery using **SQL**.



Step 1 — Building the Crime Scene (Database Setup)

Before solving a case, detectives gather evidence. In SQL, that means building tables to store facts.

We create:



employees → who works in the museum



gallery_access → map of areas



security_logs → every entry and exit

```
-- Create our Museum Database
CREATE DATABASE Pixel_Art_Museum;
USE Pixel_Art_Museum;

-- Employees
CREATE TABLE employees (
    badge_id INT PRIMARY KEY,
    name VARCHAR(100),
    role VARCHAR(100),
    salary INT
);

-- Insert initial data
INSERT INTO employees VALUES
(101, 'Sarah', 'Curator', 75000),
(102, 'Vicky', 'DB Admin', 82000),
(104, 'Elena', 'IT Specialist', 71000);
```

Now the scene is ready.



Step 2 — Where Did the Crime Happen?

The system reported the deletion happened in **Area A3**. But what does that mean?

SQL Query

```
SELECT * FROM gallery_access WHERE area_id = 'A3';
```



Output

AREA_ID	AREA_NAME
A3	Server Room

 **Conclusion:** The crime happened in the Server Room — the digital heart of the museum.

Step 3 — Who Was There at 10:15 PM?

Now we investigate the security logs to check for fingerprints.

SQL Query

```
SELECT * FROM security_logs WHERE area_id = 'A3';
```

Output

timestamp	area_id	badge_id	action
2026-02-12 22:05:00	A3	104	entry
2026-02-12 22:10:00	A3	102	entry
2026-02-12 22:12:00	A3	102	exit
2026-02-12 22:20:00	A3	104	exit

Detective Analysis

- **Vicky (102)** entered at 10:10 PM and exited at **10:12 PM**.
- **Elena (104)** entered at 10:05 PM and stayed until **10:20 PM**.

 **At 10:15 PM, only Elena was inside.**  **Primary suspect identified.**

Step 4 — The “1 Rupee” Revenge

When confronted, Elena tried to damage the system using a dangerous SQL mistake.

Malicious Command

```
UPDATE employees SET salary = 1;
```

 **No WHERE clause means every row gets updated.**

What the table looked like after sabotage

badge_id	name	role	salary
101	Sarah	Curator	1
102	Vicky	DB Admin	1
104	Elena	IT Specialist	1

A single missing word caused total chaos.

Step 5 — The Hero Fixes the Damage

The Database Admin acted fast, using surgical precision.

Recovery Command

```
UPDATE employees
SET salary = 82000
WHERE badge_id = 102;
```



Output (After Fix)

badge_id	name	role	salary
101	Sarah	Curator	1
102	Vicky	DB Admin	82000 ✓
104	Elena	IT Specialist	1



This demonstrates the power of **WHERE** — targeting exactly one record.



Key Lessons from the Mystery



1. Data tells a story

Security logs reveal timelines more accurately than human memory.



2. SQL is detective work

Every query narrows down the list of suspects and uncovers the truth.



3. NEVER run UPDATE without WHERE

One missing condition can affect the entire organization's database.



4. Logs = digital fingerprints

Even invisible digital actions leave behind traces that can be queried.

Final Verdict

The mystery of the **Pixel Mona Lisa** was solved not with fingerprints or cameras — but with **SQL queries** and logical deduction.

The museum restored order, the thief was caught, and a new rule was added:

 **Always double-check your SQL before pressing Enter.**

Want to Try It Yourself?

Run these commands in your own database terminal and become your own **Database Detective**. Compatible with: MySQL Workbench, MariaDB, phpMyAdmin, and SQLite.